Thiết kế hệ thống code đa Agent: module và bảng Option

Oct 29, 2026 · @Salmon

Mục tiêu và phạm vi

Hệ thống là một trung tâm điều phối agent code, trong đó mỗi tính năng là một module mà người dùng bật hoặc tắt trong Setting, vì không ai cần đủ 43 tính năng. Người dùng giao việc, một agent PM chia task, các agent worker (Claude, Gemini, Codex…) làm trong nhánh git riêng, và mọi thứ hiện trên một dashboard.

Phạm vi bản này là 43 tính năng trong 8 nhóm: 41 tính năng bạn chọn ban đầu và 2 tính năng bổ sung sau (đọc báo cáo bằng giọng nói, vùng đệm duyệt trước khi áp dụng). 23 tính năng còn lại chưa làm, nhưng kiến trúc module cho phép thêm sau mà không sửa lõi.

Ba tiêu chí để biết thiết kế đạt:

- Người dùng mới chạy được task đầu tiên chỉ với module lõi: đăng nhập, một subscription, một task, một worker.
- Tắt một module thì không còn tiến trình nền, bảng dữ liệu hay mục menu nào của nó hoạt động.
- Thêm một module mới chỉ cần một thư mục và một file khai báo, không sửa lõi.

Nguyên tắc thiết kế

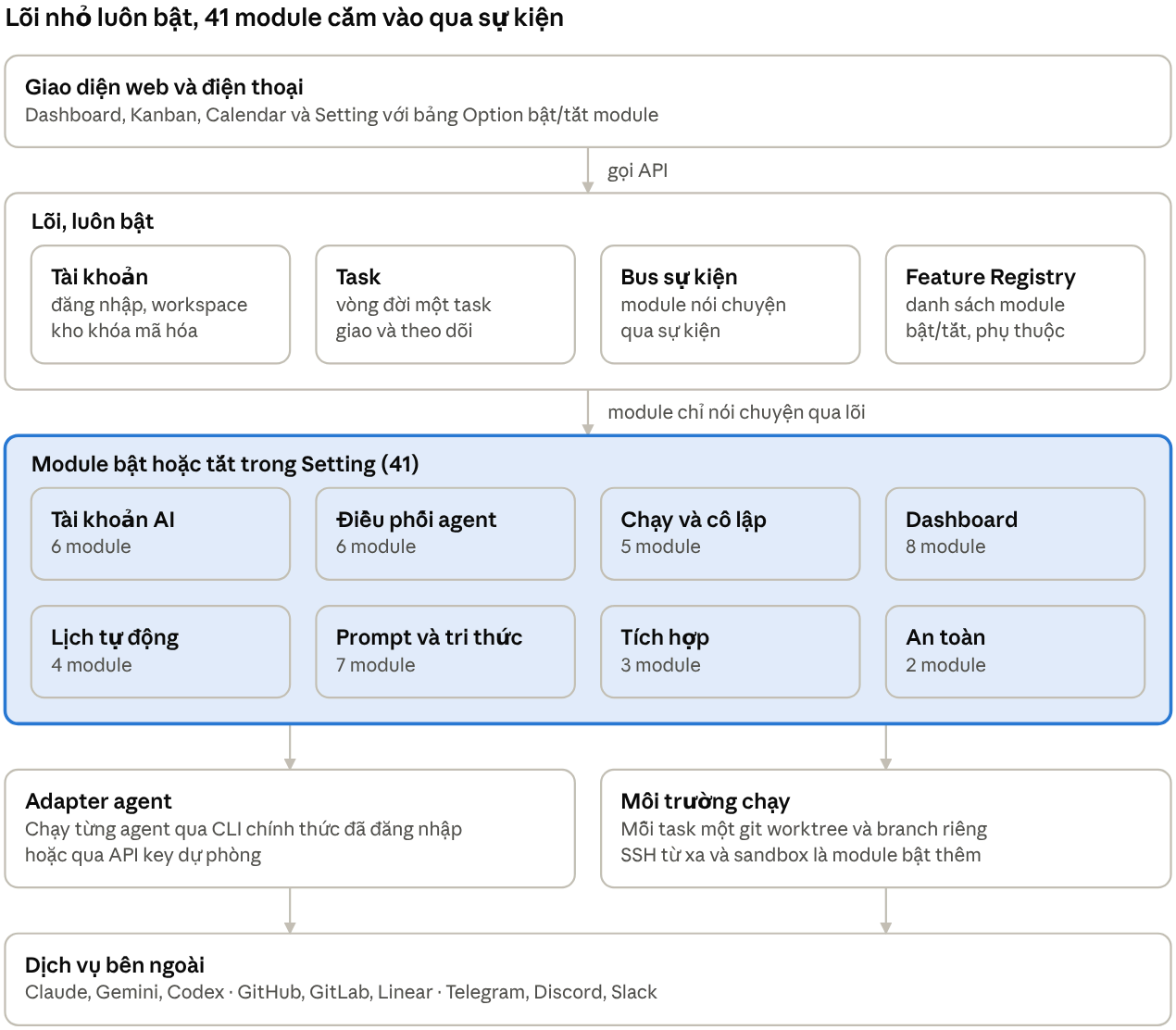
Mỗi tính năng là một module độc lập: có file khai báo riêng, tự đăng ký vào lõi khi được bật và tự gỡ khỏi hệ thống khi bị tắt. Năm nguyên tắc giữ cho điều đó đúng:

- **Lõi nhỏ và luôn bật.** Lõi chỉ gồm tài khoản, workspace, task, adapter agent, bus sự kiện, feature registry và kho khóa mã hóa.
- **Module không gọi nhau trực tiếp.** Chúng nói chuyện qua sự kiện và API của lõi, nên tắt một module không làm vỡ module khác.
- **Phụ thuộc được khai báo, không đoán.** Bật A cần B thì bảng Option bật kèm B sau khi hỏi bạn. Tắt B thì hiện danh sách module sẽ mất chức năng.
- **Tắt là tắt thật.** Dừng tiến trình nền, ẩn menu và màn hình, từ chối API của module đó. Dữ liệu cũ được giữ, không xóa.

- **Mặc định an toàn.** Module mới cài ở trạng thái tắt. Module có rủi ro như SSH từ xa, sandbox và tự merge luôn cần xác nhận khi bật.

Kiến trúc tổng thể

Hệ thống có một lõi nhỏ luôn bật, và 43 module cắm vào lõi qua sự kiện chứ không gọi nhau trực tiếp.



kiến trúc · lõi, 8 nhóm module, adapter và dịch vụ ngoài

Giao diện chỉ nói chuyện với lõi. Các module dùng chung adapter agent và môi trường chạy, nên module không cần biết agent nào đang làm việc, và thêm một nhà cung cấp AI mới chỉ cần thêm một adapter.

Cơ chế bật/tắt

Feature registry là nơi duy nhất quyết định một module có chạy hay không. Nó đọc file khai báo của từng module, lưu trạng thái bật/tắt theo phạm vi và phát sự kiện khi trạng thái đổi.

Mỗi module có một file khai báo như sau:

```
{
  "id": "cost-tracking",
  "group": "dashboard",
  "name": "Token và chi phí theo task",
  "requires": ["live-status"],
  "scopes": ["workspace", "project"],
  "effort": "medium",
  "risk": "low",
  "settings": { "budgetPerTask": { "type": "number", "default": null } }
}
```

Phạm vi. Một module có thể bật ở ba cấp: hệ thống, workspace, dự án. Cấp hẹp hơn ghi đè cấp rộng hơn, và quản trị viên có thể khóa để dự án không tự bật được.

Khi bật một module

1. Registry tính các module phụ thuộc, kể cả phụ thuộc bắc cầu, rồi hiện danh sách sẽ bật kèm.
2. Chạy migration dữ liệu của module nếu có.
3. Đăng ký route, menu, màn hình và worker nền.
4. Phát sự kiện `module.enabled` để các module khác phản ứng.

Khi tắt một module

1. Registry tìm các module đang cần nó và hiện cảnh báo, chỉ tắt tiếp khi bạn xác nhận.
2. Dừng worker nền và hủy lịch đang chờ của module.
3. Gỡ route, menu và màn hình, API trả về lỗi "module đang tắt".
4. Giữ nguyên dữ liệu và cấu hình, rồi phát sự kiện `module.disabled`.

Chu trình phụ thuộc (A cần B, B cần A) bị chặn ngay khi build, không chờ đến lúc chạy.

Danh mục 43 module

Bảng dưới liệt kê từng module bạn đã chọn, module nào phải bật trước nó, và công sức triển khai (ước lượng của tôi). Cột "Cần bật trước" ghi "lõi" khi chỉ cần các thành phần luôn bật.

Nhóm	Module	Cần bật trước	Công sức
1. Tài khoản AI	Đăng ký / đăng nhập hệ thống	lõi	Nhẹ
1. Tài khoản AI	Kết nối subscription qua CLI chính thức	Đăng nhập	Vừa
1. Tài khoản AI	Nhiều tài khoản cùng một nhà cung cấp	Kết nối subscription	Vừa
1. Tài khoản AI	Theo dõi hạn mức từng tài khoản	Kết nối subscription	Vừa
1. Tài khoản AI	Tự chuyển tài khoản khi hết hạn mức	Nhiều tài khoản, Theo dõi hạn mức	Vừa
1. Tài khoản AI	API key trả theo token làm dự phòng	Đăng nhập, kho khóa của lõi	Nhẹ
2. Điều phối agent	Chọn Lead (PM) và Worker cho từng task	Kết nối subscription	Vừa
2. Điều phối agent	Agent viết tài liệu tự chạy khi task xong	Chọn Lead và Worker	Vừa
2. Điều phối agent	Agent test chuyên trách	Chọn Lead và Worker	Vừa
2. Điều phối agent	PM tự phân rã mục tiêu thành đồ thị task	Chọn Lead và Worker	Nặng
2. Điều phối agent	Cổng duyệt của con người	lõi	Vừa
2. Điều phối agent	Kênh nhắn tin giữa các agent	Chọn Lead và Worker	Nặng
3. Chạy và cô lập	Mỗi task một git worktree và branch riêng	lõi	Vừa

Nhóm	Module	Cần bật trước	Công sức
3. Chạy và cô lập	Tạo PR với mô tả do AI viết, merge khi đạt điều kiện	Git worktree, Nối GitHub/GitLab/Linear	Vừa
3. Chạy và cô lập	Chạy trên máy từ xa qua SSH	Git worktree	Nặng
3. Chạy và cô lập	Tự host, chạy 24/7 trên server	lõi (cách triển khai)	Vừa
3. Chạy và cô lập	Workspace nhiều repo	Git worktree	Vừa
4. Dashboard	Bảng task dạng Kanban theo trạng thái	lõi	Vừa
4. Dashboard	Xem agent đang làm gì theo thời gian thực	lõi (bus sự kiện)	Vừa
4. Dashboard	Dự tính thời điểm xong của task	Xem thời gian thực, Lịch sử từng lần chạy	Nặng
4. Dashboard	Báo cáo tổng kết ngày và tuần	Kanban, Lịch sử từng lần chạy	Vừa
4. Dashboard	Đọc báo cáo bằng giọng nói	Báo cáo tổng kết ngày và tuần	Nhẹ
4. Dashboard	Token và chi phí theo task, theo agent	Xem thời gian thực	Vừa
4. Dashboard	Xem diff ngay trong giao diện và mở bản xem trước	Git worktree	Vừa
4. Dashboard	Xem lại lịch sử từng lần chạy	lõi	Vừa
4. Dashboard	Cây quan hệ PM và agent con	Chọn Lead và Worker, Xem thời gian thực	Nhẹ
5. Lịch tự động	Calendar để lên lịch task định kỳ	Kết nối subscription	Vừa
5. Lịch tự động	Job viết bằng file Markdown	Calendar	Nhẹ
5. Lịch tự động	Khung giờ cấm chạy	Calendar	Nhẹ
5. Lịch tự động	Hàng đợi chạy qua đêm	Calendar	Vừa

Nhóm	Module	Cần bật trước	Công sức
6. Prompt và tri thức	Thư viện prompt gắn theo agent và theo skill	Chọn Lead và Worker	Vừa
6. Prompt và tri thức	Hồ sơ kỹ thuật dự án	Thư viện prompt	Vừa
6. Prompt và tri thức	Skill theo chuẩn SKILL.md	Thư viện prompt	Vừa
6. Prompt và tri thức	Đồng bộ file hướng dẫn từ một nguồn	Hồ sơ kỹ thuật, Skill SKILL.md	Vừa
6. Prompt và tri thức	Bộ nhớ dùng chung giữa các agent	Thư viện prompt	Nặng
6. Prompt và tri thức	Tự ghi bài học sau mỗi task	Bộ nhớ dùng chung	Nhẹ
6. Prompt và tri thức	Đồ thị tri thức của code	Bộ nhớ dùng chung	Nặng
7. Tích hợp	Nối GitHub, GitLab, Linear	Đăng nhập	Vừa
7. Tích hợp	Thông báo qua Telegram, Discord, Slack	lõi (bus sự kiện)	Vừa
7. Tích hợp	Duyệt và theo dõi từ điện thoại	Đăng nhập, Cổng duyệt, Xem thời gian thực	Vừa
8. An toàn	Nhật ký kiểm toán	lõi (bus sự kiện)	Nhẹ
8. An toàn	Sandbox cho task rủi ro	Git worktree	Nặng
8. An toàn	Vùng đệm duyệt trước khi áp dụng (đóng băng code, chỉ áp dụng sau khi bạn duyệt)	Git worktree, Cổng duyệt, Xem diff	Vừa

Bảng Option trong Setting

Bảng Option là một tab "Tính năng" trong Setting, nơi bật hoặc tắt từng module mà không phải đụng vào cấu hình kỹ thuật. Mọi thay đổi áp dụng ngay, không cần khởi động lại, trừ khi module ghi rõ "cần khởi động lại worker".

Bố cục

- Trên cùng là bộ chọn phạm vi (Hệ thống, Workspace, Dự án), ô tìm kiếm và bộ lọc "Đang bật" / "Đang tắt".
- Ngay dưới là hàng gói mẫu để bật cả bộ chỉ bằng một lần bấm.
- Tiếp theo là 8 nhóm, mỗi nhóm thu gọn được và hiện "đang bật 3/6".
- Mỗi module là một hàng gồm công tắc, tên, mô tả một dòng, nhãn công sức và rủi ro, và biểu tượng bánh răng mở cấu hình riêng. Bánh răng chỉ sáng khi module đang bật.

Hành vi khi bấm công tắc

- Bật module cần module khác: hộp thoại "Sẽ bật kèm: A, B" với nút Xác nhận và Hủy.
- Tắt module đang được module khác cần: hộp thoại "Tắt sẽ làm mất chức năng: X, Y". Bạn có thể tắt cả chuỗi hoặc hủy.
- Bật module có rủi ro (SSH, sandbox, tự merge): hiện một dòng giải thích rủi ro và yêu cầu xác nhận.
- Module bị khóa ở cấp cao hơn: công tắc mờ đi, kèm dòng "Do quản trị viên workspace quy định".
- Mọi lần đổi ghi vào nhật ký, và có nút Hoàn tác thay đổi gần nhất.

Gói mẫu đề xuất. Đây là gợi ý để bạn chỉnh, không phải danh sách cố định.

Gói	Dành cho	Module bật
Solo nhẹ	Một người, một dự án	Đăng nhập, Kết nối subscription, Chọn Lead và Worker, Git worktree, Kanban, Xem thời gian thực, Thư viện prompt, Hồ sơ kỹ thuật
Nhóm nhỏ	Vài người, nhiều tài khoản AI	Gói Solo nhẹ, cộng Nhiều tài khoản, Theo dõi hạn mức, Tự chuyển tài khoản, Agent test, Agent viết tài liệu, Diff, Tạo PR, Nối GitHub/GitLab/Linear, Cổng duyệt, Calendar, Chi phí, Thông báo
Đầy đủ	Vận hành tự động nhiều dự án	Cả 43 module

Mô hình dữ liệu

Trạng thái bật/tắt nằm trong bốn bảng nhỏ, và trạng thái hiệu lực của một module được tính từ bảng cài đặt chứ không lưu thẳng.

Bảng	Khóa chính	Nội dung
feature_module	id module	Danh mục module, nạp từ file khai báo: nhóm, tên, requires, scopes, risk, effort, phiên bản
feature_setting	module, loại phạm vi, id phạm vi	enabled, locked, config (JSON theo schema của module), người đổi, thời điểm đổi
feature_pack	id gói	Tên gói và danh sách module của gói
feature_audit	id sự kiện	Module, phạm vi, hành động bật/tắt/đổi cấu hình, người thực hiện, lý do, thời điểm

Trạng thái hiệu lực của một module tại một dự án được tính theo thứ tự:

1. Nếu cấp hệ thống hoặc workspace đặt `locked`, dùng giá trị của cấp đó.
2. Nếu không, dùng giá trị ở cấp dự án nếu có.
3. Nếu không, dùng giá trị ở cấp workspace, rồi cấp hệ thống.
4. Nếu không có giá trị nào, mặc định là tắt.

Quyền đổi trạng thái theo cấp: chủ hệ thống đổi ở cấp hệ thống, quản trị viên workspace đổi ở workspace, chủ dự án đổi ở dự án. Ai đổi cũng bị ghi vào feature_audit, kể cả khi module Nhật ký kiểm toán đang tắt.

Thứ tự triển khai

Làm lõi và bảng Option trước, rồi thêm module theo thứ tự phụ thuộc, để mỗi giai đoạn đều chạy được và dùng được. Giai đoạn 0 dựng lõi cùng màn hình Option, nên từ module đầu tiên đã có công tắc bật/tắt.

Giai đoạn	Module	Cổng để sang giai đoạn sau
0. Lõi	Đăng nhập, cộng lõi: workspace, task, adapter agent, bus sự kiện, feature registry, kho khóa, màn hình Option	Bật và tắt được một module mẫu, dữ liệu còn nguyên sau khi tắt

Giai đoạn	Module	Cổng để sang giai đoạn sau
1. Chạy được một task	Kết nối subscription, Chọn Lead và Worker, Git worktree, Kanban, Xem thời gian thực, Thư viện prompt, Hồ sơ kỹ thuật	Giao một task cho worker, xem agent chạy và nhận kết quả trên nhánh riêng
2. Vòng làm việc đầy đủ	Agent test, Agent viết tài liệu, Diff và xem trước, Lịch sử từng lần chạy, Tạo PR, Nối GitHub/GitLab/Linear, Cổng duyệt, Nhiều tài khoản, Theo dõi hạn mức, Tự chuyển tài khoản, API key dự phòng, Chi phí	Một dự án thật chạy trọn một tuần chỉ bằng hệ thống
3. Tự động và tri thức	Calendar, Job Markdown, Khung giờ cấm, Hàng đợi qua đêm, Báo cáo ngày và tuần, Dự tính thời điểm xong, Skill SKILL.md, Đồng bộ file hướng dẫn, Bộ nhớ chung, Tự ghi bài học, Thông báo, Mobile, Nhật ký kiểm toán	Hệ thống chạy qua đêm và để lại báo cáo buổi sáng
4. Nặng và rủi ro	PM phân rã đồ thị task, Kênh nhắn tin giữa agent, SSH từ xa, Tự host 24/7, Workspace nhiều repo, Sandbox, Đồ thị tri thức, Cây PM và agent con	Từng module qua kiểm thử riêng trước khi cho bật ở chế độ thường

Giai đoạn 3 chỉ có số liệu thật để dự tính thời điểm xong sau khi Lịch sử từng lần chạy đã tích lũy đủ dữ liệu từ giai đoạn 2.

Rủi ro và quyết định cần chốt

Có bốn điểm cần bạn quyết trước khi bắt đầu giai đoạn 0, vì chúng đổi hình dạng của lõi.

- [ ] **Kho khóa mã hóa có vào lõi không?** Bạn không chọn "Quản lý secret", nhưng module API key dự phòng bắt buộc phải lưu khóa. Tôi đề xuất đưa kho khóa vào lõi thay vì làm module.
- [ ] **Chỉ chạy CLI chính thức, hay có đường khác?** Thiết kế này chỉ dùng CLI chính thức đã đăng nhập sẵn. Dùng token OAuth ngoài CLI có thể vi phạm điều khoản của nhà cung cấp, nên tôi không đưa vào.

- [ ] **Hệ thống chạy ở đâu?** Trên máy người dùng như Vibe Kanban, hay trên server như module Tự host? Chọn khác nhau thì cách đăng nhập CLI subscription khác nhau, vì CLI cần đăng nhập ở nơi nó chạy.
- [ ] **Giai đoạn 4 làm trước hay sau?** PM phân rã đồ thị task là điểm khác biệt lớn nhất của bạn, nhưng cũng nặng nhất. Làm sớm thì có giá trị sớm, làm muộn thì nền vững hơn.

Hai rủi ro kỹ thuật cần theo dõi:

- **Số liệu hạn mức không đồng nhất.** Mỗi nhà cung cấp trả thông tin hạn mức khác nhau, và có nhà không trả. Module Theo dõi hạn mức có thể chỉ ước lượng được từ lịch sử dùng.
- **Dự tính thời điểm xong chưa có mẫu sẵn.** Tôi chưa thấy dự án mã nguồn mở nào làm tốt phần này, nên cần thử nghiệm riêng sau khi có dữ liệu thật.